김승완 Seungwan Kim

I work on deployment automation and monitoring on AWS and Kubernetes.

Over seven months in Megazone Cloud's MSP Solutions Architect program I built five projects. I started with a 3-tier EC2 setup, moved to serverless and to networking for on-prem Kubernetes, then ran CI/CD and monitoring for a team on EKS. There I kept seeing deploys pass while the service returned 5xx. So in the last project I built, on my own, a pipeline that reads metrics right after each deploy and makes its own pass or fail call.

AWS SAA-C03 Linux Master Lv.2 TOEIC 960 Korean & English, native

seungwankim364@gmail.com GitHub Notion

Projects

When	Project	Team	My part
2026.05 ~ 07	**Quality Gate**	Solo	Everything
2026.05 ~ 07	**G2MPT**	4	CI/CD, monitoring
2026.03	**Fullcount**	4	Network, infrastructure
2026.02	**Photoshoot Blog**	Solo	Full-stack, AWS
2026.01	**MyFoodMap**	3	AWS infrastructure

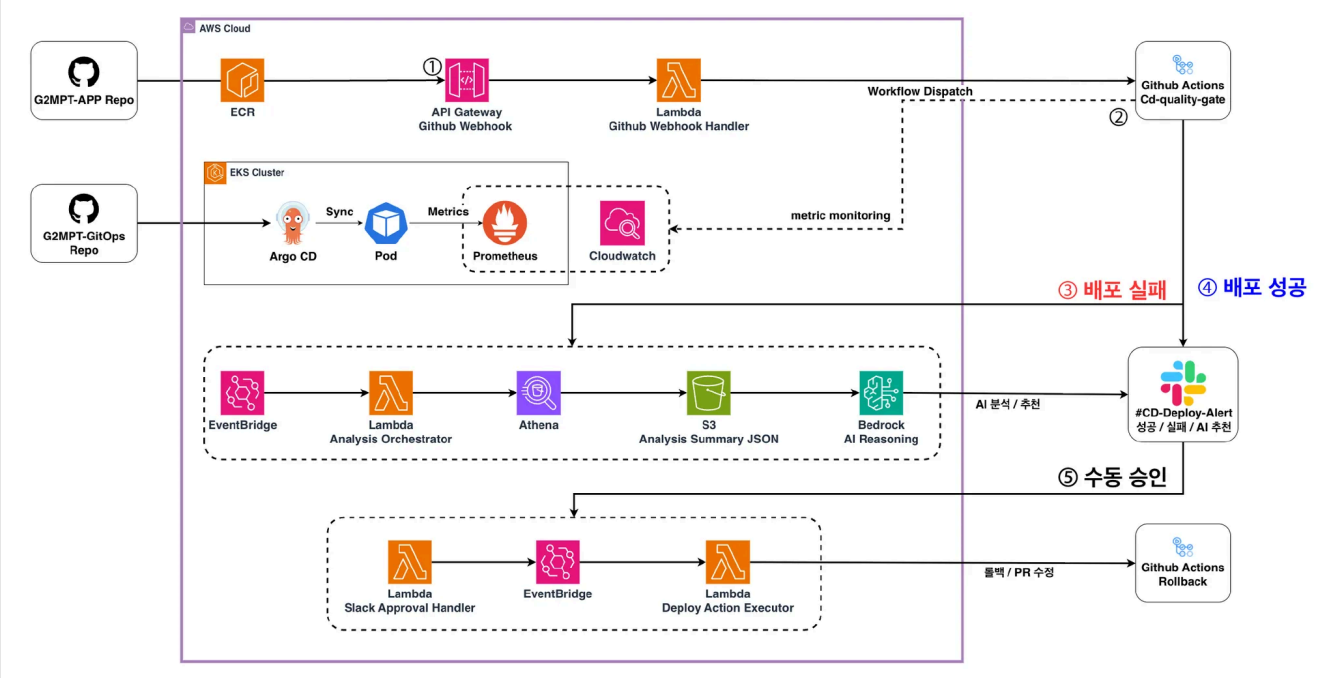
Quality Gate

CD quality gate with AI-assisted failure analysis

Solo 2026.05.11 ~ 07.08

GitHub Actions, Argo CD, Prometheus, CloudWatch, EventBridge, Lambda, Athena, Bedrock, Terraform

On G2MPT I would see GitHub Actions green and Argo CD Synced while the service returned 5xx. The pipeline only checked that the deploy job finished. So I added a step that watches real metrics for five minutes after each deploy and makes a second pass or fail call.



From post-deploy checks to Slack approval and rollback

Checks live in a separate repo

Inside the deploy workflow the check would add five minutes to every deploy, and a broken check would block releases. A webhook calls a workflow in another repo once the deploy ends.

Prometheus and CloudWatch together

The app can answer fine while RDS runs out of connections, so one source is not enough. Error rate, latency and pod restarts come from Prometheus, and RDS and SQS come from CloudWatch.

Rollback waits for Slack approval

I first planned to roll back automatically. One false positive reverting a good release is worse than the original problem, so Bedrock writes the likely cause and a suggested action, and the rollback runs only after someone presses the button in Slack.

Result

- Five minutes after each deploy, error rate, latency, pod restarts, RDS connections and SQS backlog decide pass or fail
- On failure, Athena pulls logs from that window and Bedrock's analysis goes to Slack with Grafana and Argo CD links
- Rollback, Manual Fix and Change Apply each run from a single approval button
- Four Lambdas plus EventBridge, Athena and API Gateway, all in Terraform, costing next to nothing between deploys

Open question

I have not measured how accurate the verdicts are. Next step is to replay good and deliberately broken deploys to get a false-positive rate, then let only proven cases run without approval.

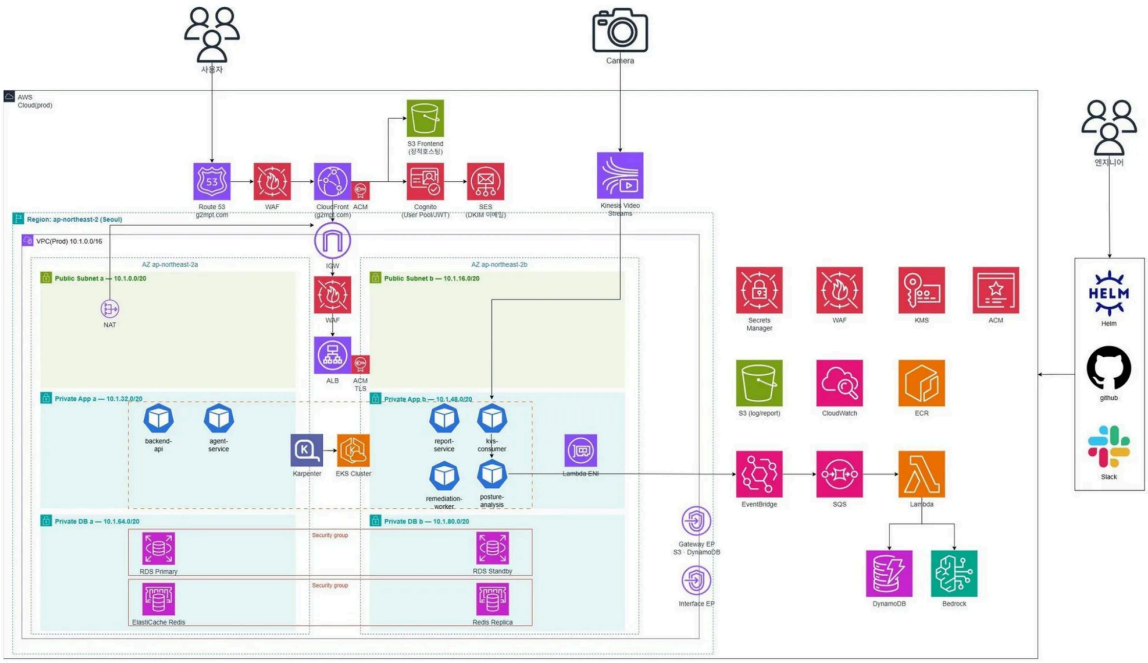
G2MPT

AI personal-training platform on AWS EKS

Team of 4 2026.05.11 ~ 07.08 CI/CD and monitoring

GitHub Actions, Argo CD, Helm, EKS, Karpenter, Prometheus, Grafana, CloudWatch

We moved a service that ran its API, AI, video analysis and reports on one on-prem server onto microservices on EKS. My job was to replace manual deploys and give the team the logs and metrics it never had. With nine services, automating deploys alone would have let each one ship in its own shape, so I set the deploy standard and the monitoring together.



Production layout in ap-northeast-2

Argo CD does the deploy

GitHub Actions pushes the image to ECR and bumps the tag in the GitOps repo. Argo CD applies it, so CI never holds cluster credentials, and AWS access moved to OIDC so no access keys sit in the repo.

One Helm chart

Ten resource types, including Deployment, HPA, PDB and ServiceMonitor, live in one shared template, and each service only changes its values. Because ServiceMonitor is in the template, a new service gets Prometheus scraping without extra work.

Two alert paths

RDS, SQS, ElastiCache and KVS sit outside the cluster where Prometheus cannot see them. Pods and nodes alert through Alertmanager, AWS resources through CloudWatch, SNS and Chatbot, both into the same Slack channel.

Result

- Nine services on nine pipelines, 412 workflow runs over the project and about 2 min 30 s per deploy
- All nine deploy only through GitOps, and Argo CD reverts any manual kubectl change back to what Git says
- Six alert rule files kept in one place, covering teammates' SQS and KVS too, plus rules that fire if Alertmanager itself goes down

What I learned

Tuning resource requests showed me those values drive the cost of the whole cluster. Karpenter launches nodes from requests, so generous values leave idle nodes piling up. After that I tuned them together with our infra teammate.

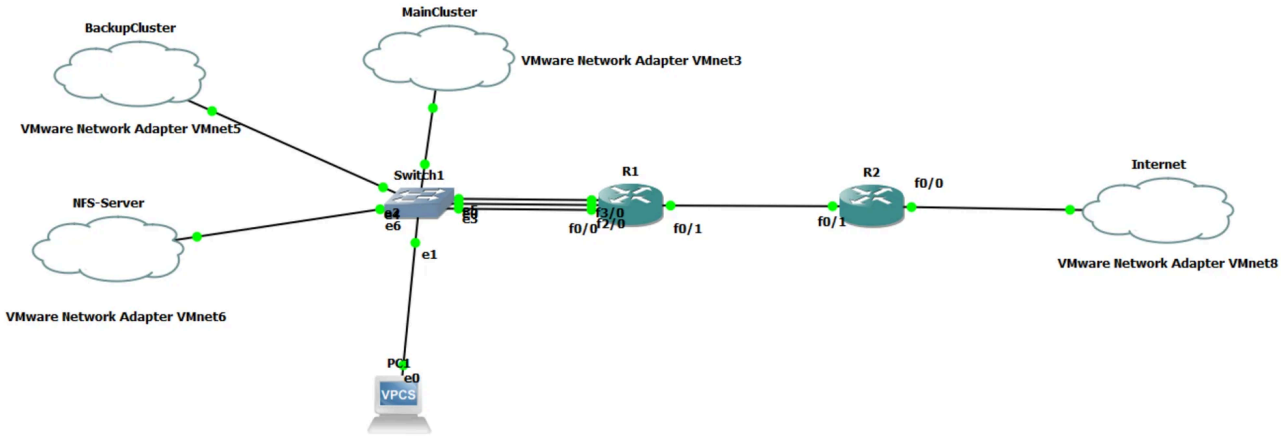
Fullcount

Live MLB play-by-play on on-prem Kubernetes

Team of 4 2026.03.10 ~ 03.23 Network and infrastructure

Kubernetes, GNS3, BGP, MetalLB, NAT/PAT, ACL, Tailscale, NFS, cert-manager

What an ALB does on AWS had to be built here from the routers up. I took the network part to see what the cloud had been doing for me.



Network topology built in GNS3

VIPs over BGP

Edge, core and MetalLB got AS65002, 65001 and 65003, and MetalLB advertises VIPs over eBGP. If a node dies only its route drops and traffic moves to the others.

No NAT for service traffic

NAT on inbound traffic hides client IPs and puts all load on one edge router. NAT/PAT handles management outbound only. Service traffic comes straight in on the VIP routes.

Tailscale instead of a bastion

A bastion is one more public IP to defend. The office network also blocked direct traffic between our machines, so a Tailscale overlay with subnet routing for the VIP range solved both.

Result

- Six network segments, nine nodes, two routers, three ASes
- Shutting the main cluster interface on the core router withdrew its BGP route and traffic reached the backup cluster
- Nine blockers logged by layer, such as DNS port 53 missing from the ACL, a missing Tailscale subnet route, Flannel not applied

What I learned

One missing ACL line broke DNS, and one missing subnet route kept us off the VIPs. It was the first time I saw which layer a single security-group rule had been covering. Now I start network problems by finding the layer where it breaks.

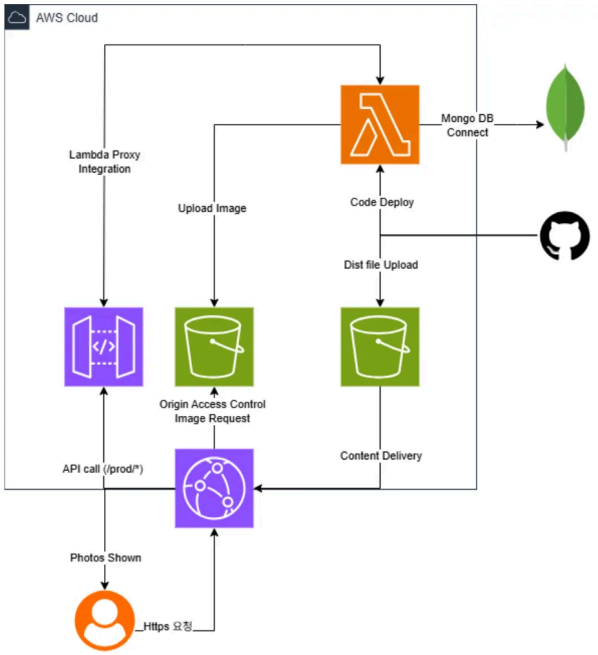
Photoshoot Blog

Serverless photo blog on AWS

Solo 2026.02.10 ~ 02.20

S3, CloudFront, API Gateway, Lambda, MongoDB Atlas, React, GitHub Actions

On MyFoodMap a single t3.micro cost over $10 a month even with nobody visiting. A photo album I open now and then did not need a server running, so this time I built it with none.



One CloudFront splits static files, API and images

One CloudFront entry

Static files, the /prod/* API and images are split by path in CloudFront. One origin meant no CORS setup, and both S3 buckets sit behind OAC so nobody reaches them by bucket URL.

Reused DB connections

Lambda scales out under load, and opening a connection inside the handler each time hits the Atlas limit fast. The connection lives outside the handler and is reused while warm.

Separate frontend and Lambda deploys

A UI fix should not redeploy the backend, so there are two pipelines. The frontend one ends with a CloudFront invalidation so changes show up right away.

Result

- No always-on server, so with no requests the cost is close to zero
- Sign-up, JWT login, photo upload, edit, delete and date sorting built and shipped alone in 11 days
- Frontend and Lambda both deploy through GitHub Actions

If I built it again

Uploads still go through Lambda. Given API Gateway's payload limit and Lambda cost, presigned URLs straight to S3 fit better, and that is the first thing I would change.

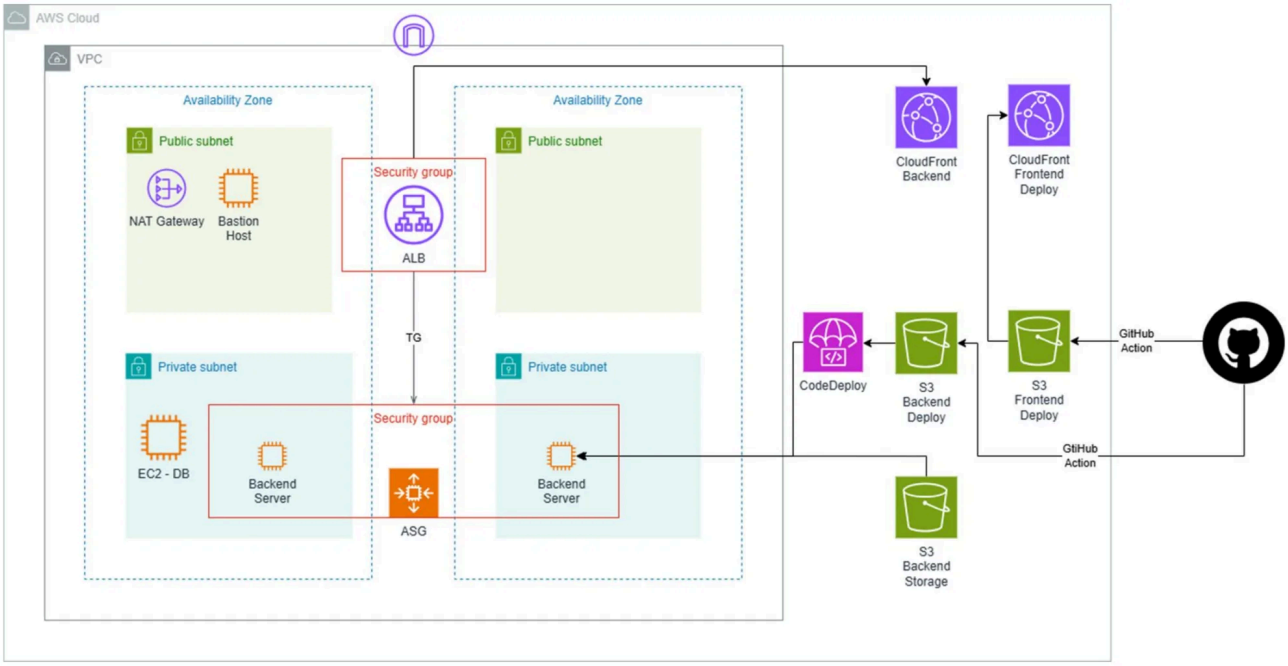
MyFoodMap

3-tier restaurant journal on AWS

Team of 3 2026.01.26 ~ 01.29 AWS infrastructure

S3, CloudFront, ALB, ASG, CodeDeploy, React, Node.js, MySQL

A map of the restaurants you have been to, with your own reviews. At first frontend, backend and DB each ran on one EC2 instance, so any single failure stopped the service, and deploys were manual.



After adding ALB, ASG and split subnets

Frontend to S3

Moved the frontend from EC2 and Nginx to S3 and CloudFront, with OAC and HTTPS.

Backend across two AZs

The backend runs in an Auto Scaling Group behind an ALB across two AZs, with public and private subnets so the app and DB cannot be reached from outside.

Images to S3

Review photos on an EC2 disk are invisible to a newly launched instance. The ASG only worked once images moved to S3.

Result

- Frontend ships through GitHub Actions to S3 and CloudFront and the backend through S3 and CodeDeploy, so nothing is deployed by hand
- If one backend instance dies, the other AZ keeps serving
- From the first setup to the revised one in four days

What I learned

Only after adding the ASG did I see why images had to be in S3. Since then I decide where state lives before laying out servers. The DB still sits on one EC2 instance, and moving it to RDS Multi-AZ is what is left.

Experience & education

2025.12 ~ 2026.07

Megazone Cloud, MSP Solutions Architect Program

- Linux and networking fundamentals, AWS architecture design
- Containers on Docker and Kubernetes, IaC with Terraform
- CI/CD with GitHub Actions and Jenkins, monitoring with Prometheus and Grafana
- Delivered the five projects above

2024.02 ~ 2025.08

Republic of Korea Army, K-9 Gun Section Chief

Capital Mechanized Infantry Division / Volunteered while holding overseas permanent residency

- Led a gun crew, accountable for crew safety and fire missions
- Top live-fire score in the battalion, with every mission completed without a safety incident

2023.09 ~ 2024.02

CGBIO

Human-tissue quality control, SOP operations, overseas sales

- Led meetings with overseas suppliers and closed a contract with a new one
- Checked inspection and manufacturing against SOPs, and reviewed and translated SOPs between Korean and English
- Interpreted for external audits and drafted CAPA reports

2016.09 ~ 2023.04

University of Waterloo

BSc, Honours Science / Waterloo, ON, Canada

- 2020.05 ~ 2023.08 English tutor for elementary to high-school students, from pronunciation to essay editing

Certifications & award

2026.10.02

Linux Master Level 2

Korea Association for AI & ICT Promotion

2026.09.04

AWS Certified Solutions Architect, Associate

Amazon Web Services

2025.08.04

Certificate of Honor for Military Service

Military Manpower Administration / for volunteering while a permanent resident abroad

2024.05

TOEIC 960

2023.05

Introduction to Python 3, Introduction to SQL

WatSPEED, University of Waterloo

Skills

AWS	VPC, EKS, ECR, S3, CloudFront, ALB/ASG, Lambda, API Gateway, EventBridge, Athena, Bedrock, CodeDeploy All five projects
Kubernetes	EKS and on-prem clusters, standardized Helm chart (10 resource types), HPA, PDB, NetworkPolicy, ExternalSecret G2MPT, Fullcount
CI/CD	Nine GitHub Actions pipelines, Argo CD GitOps, OIDC short-lived credentials, CodeDeploy G2MPT, Quality Gate, MyFoodMap
Monitoring	Prometheus alert rules, Grafana dashboards, Alertmanager, CloudWatch and Athena log analysis G2MPT, Quality Gate
Network	BGP/MetalLB L3 load balancing, NAT/PAT and ACL policy, GNS3 routers, Tailscale Fullcount
IaC	AWS serverless infrastructure in Terraform Quality Gate
Linux	Hosts behind on-prem Kubernetes, package management, NFS storage Linux Master Lv.2
Other	Docker, Jenkins, NFS, cert-manager, MongoDB Atlas, MySQL/Prisma, Redis Across projects

Languages	Korean (native), English (native, TOEIC 960), French (intermediate)

Contact

seungwankim364@gmail.com github.com/seungwankim364 Notion page